CareShare

Repo-backed one-page application summary

What It Is

CareShare is a web app for coordinating care around an elderly loved one. Repo copy and UI flows position it as one shared place for families to manage tasks, costs, events, care details, and collaboration.

Who It's For

Primary persona: family caregivers coordinating responsibilities with relatives. The repo also includes an admin portal for care providers or nursing-home staff managing multiple families.

What It Does

- Creates family workspaces and supports multi-step onboarding plus member invitations.
- Tracks care tasks with assignments, due dates, priorities, and attachments.
- Manages shared finances: bills, contributions, pending costs, exports, and dashboards.
- Maintains a shared calendar for appointments, birthdays, visits, and deliveries.
- Stores care-planning data such as care recipient profile, medications, notes, and documents.
- Provides family collaboration tools including messages, life stories, and resource directories.
- Includes admin screens for users, families, blog/content, database tools, and settings.

How It Works

Frontend: Next.js App Router pages under *app/* render the marketing site, onboarding, family pages, dashboards, and admin UI.
Auth: NextAuth with credentials and optional Google sign-in, wrapped by a SessionProvider; middleware protects */dashboard*, */family*, and */admin* routes.
Application layer: Route handlers under *app/api/* serve family-scoped CRUD for tasks, events, costs, resources, notes, medications, documents, messages, onboarding, and admin operations.
Data layer: Prisma talks to PostgreSQL. Schema evidence shows core models for users, families, memberships, invitations, care recipients, tasks, events, costs, messages, documents, resources, care plans, scenarios, contributions, blog posts, and audit logs.
Files: The upload route writes image/PDF receipts to *public/uploads/receipts* on the app server.
Data flow: Browser page -> Next.js page/client fetch -> API route -> auth/permission checks -> Prisma -> PostgreSQL -> JSON/UI refresh.

How To Run

- Install deps: **npm install**
- Create **.env** with **DATABASE_URL**, **NEXTAUTH_URL**, and **NEXTAUTH_SECRET**. Google OAuth vars appear optional in code.
- Prepare Prisma: **npx prisma generate** and **npx prisma db push**
- Start dev server: **npm run dev**
- Open **http://localhost:3000**

Not Found In Repo

- Production monitoring/queueing/notification service wiring: Not found in repo.
- A checked-in .env.example file: Not found in repo.

Evidence Basis

Sources used: README.md, package.json, prisma/schema.prisma, lib/auth.ts, lib/auth-utils.ts, middleware.ts, app/layout.tsx, app/page.tsx, app/onboarding/page.tsx, dashboard pages, and family/admin API routes.